# Industrial Internship Report on
# “Naad Music Player”
# Prepared by
# Swar Sawalkar


*Executive Summary*

This report presents the Industrial Internship project completed under the guidance of upskill Campus and The IoT Academy in collaboration with UniConverge Technologies Pvt Ltd (UCT). The internship spanned 6 weeks, during which the project was designed, developed, and documented.

The project, titled "Naad Music Player", is a modern Java-based desktop audio application developed using Java Swing. It delivers an efficient and visually appealing platform for audio playback and management. The system supports multiple audio formats, playlist management, shuffle and repeat modes, and equalizer presets — all without relying on external libraries.

The project demonstrates object-oriented programming (OOP), event-driven architecture, and multithreading concepts, reinforcing core Java competencies in a real-world multimedia application context.

This internship provided a valuable opportunity to bridge academic learning with industry-grade software development and to strengthen problem-solving, UI design, and backend integration skills.

## TABLE OF CONTENTS


1 Preface ..... 3
2 Introduction ..... 4
2.1 About UniConverge Technologies Pvt Ltd ..... 4
2.2 About upskill Campus ..... 8
2.3 Objective ..... 9
2.4 Reference ..... 9
2.5 Glossary ..... 10
3 Problem Statement ..... 11
4 Existing and Proposed Solution ..... 12
5 Proposed Design / Model ..... 13
5.1 High Level Diagram ..... 13
5.2 Low Level Diagram ..... 13
5.3 Interfaces ..... 13
6 Performance Test ..... 14
6.1 Test Plan / Test Cases ..... 14
6.2 Test Procedure ..... 14
6.3 Performance Outcome ..... 14
7 My Learnings ..... 15
8 Future Work Scope ..... 16

# 1 Preface

This report summarizes the 6-week industrial internship completed under the upskill Campus program, in association with The IoT Academy and UniConverge Technologies Pvt Ltd (UCT). The internship offered an immersive, hands-on experience bridging academic knowledge with real-world software engineering.

In today's competitive job market, practical exposure is indispensable for engineering graduates. While theoretical concepts form the backbone of education, internships like this one serve as the critical link that transforms classroom learning into production-ready skills. This project — the Naad Music Player — was designed to reflect that ethos: solving a real technical problem using core Java technologies, without depending on external libraries or bloated frameworks.

The Naad Music Player is a fully functional, cross-platform desktop audio application developed using Java Swing and the Java Sound API. The application supports multiple audio formats, offers playlist management, shuffle and repeat modes, volume control, and equalizer presets — all wrapped in a modern, visually appealing graphical user interface.

The project was structured over six weeks as follows:

- Week 1: Problem analysis, requirement gathering, and setting up the development environment.
- Week 2: Designing the system architecture and creating UI wireframes.
- Week 3: Backend development — audio engine using Java Sound API, multithreading, and data structures.
- Week 4: Frontend development — Java Swing UI components, event handling, and integration with backend.
- Week 5: Testing, bug fixing, and performance optimization.
- Week 6: Documentation, report writing, and final code submission.

This internship gave me the opportunity to work on a self-contained software project from ideation to delivery. The guidance provided by the UCT team and the structured framework of upskill Campus enabled me to develop not just technical skills, but also professional discipline, time management, and the ability to document and communicate technical work effectively.

I extend my heartfelt gratitude to the entire upskill Campus and UCT team for creating this platform and providing meaningful work. My special thanks to the project mentors, the IoT Academy faculty, and my peers who supported and encouraged me throughout this journey.

To my juniors and peers: embrace every internship opportunity with full commitment. The skills you build here — writing clean code, debugging systematically, and documenting your work — will differentiate you in your career. A technical project that is well-documented is worth far more than code that only you understand.

# 2 Introduction

## 2.1 About UniConverge Technologies Pvt Ltd

A company established in 2013 and working in Digital Transformation domain and providing Industrial solutions with prime focus on sustainability and RoI.

For developing its products and solutions it is leveraging various Cutting Edge Technologies e.g. Internet of Things (IoT), Cyber Security, Cloud computing (AWS, Azure), Machine Learning, Communication Technologies (4G/5G/LoRaWAN), Java Full Stack, Python, Front end etc.

**UCT IoT Platform**

UCT Insight is an IOT platform designed for quick deployment of IOT applications on the same time providing valuable "insight" for your process/business. It has been built in Java for backend and ReactJS for Front end. It has support for MySQL and various NoSql Databases.

- It enables device connectivity via industry standard IoT protocols - MQTT, CoAP, HTTP, Modbus TCP, OPC UA
- It supports both cloud and on-premises deployments.

It has features to:

- Build Your own dashboard
- Analytics and Reporting
- Alert and Notification
- Integration with third party application (Power BI, SAP, ERP)
- Rule Engine

**Smart Factory Platform**

Factory Watch is a platform for smart factory needs. It provides Users/Factory:

- with a scalable solution for their Production and asset monitoring
- OEE and predictive maintenance solution scaling up to digital twin for your assets.
- to unleash the true potential of the data that their machines are generating and helps to identify the KPIs and also improve them.
- A modular architecture that allows users to choose the service that they want to start and then can scale to more complex solutions as per their demands.

Its unique SaaS model helps users to save time, cost and money.

**LoRaWAN Based Solution**

UCT is one of the early adopters of LoRaWAN technology and providing solution in Agritech, Smart Cities, Industrial Monitoring, Smart Street Light, Smart Water/Gas/Electricity metering solutions etc.

**Predictive Maintenance**

UCT is providing Industrial Machine health monitoring and Predictive Maintenance solution leveraging Embedded System, Industrial IoT and Machine Learning Technologies by finding Remaining Useful Lifetime of various Machines used in production process.

## 2.2 About upskill Campus

Upskill Campus along with The IoT Academy and in association with Uniconverge Technologies has facilitated the smooth execution of the complete internship process.**USC is a career development** platform that delivers personalized executive coaching in a more affordable, scalable and measurable way.Seeing need of upskilling in self paced manner along-with additional support services e.g. Internship, projects, interaction with Industry experts, Career growth Services.

**The IoT Academy**

The IoT Academy is the EdTech Division of UCT that is running long executive certification programs in collaboration with EICT Academy, IITK, IITR and IITG in multiple domains.

## 2.3 Objective

The objective for this internship program was to:

- Get practical experience of working in the industry.
- To solve real world problems.
- To have improved job prospects.
- To have Improved understanding of our field and its applications.
- To have Personal growth like better communication and problem solving.

## 2.4 Reference

[1] Oracle Java Documentation — https://docs.oracle.com/en/java/

[2] Java Swing Tutorial — https://docs.oracle.com/javase/tutorial/uiswing/

[3] Java Sound API Guide — https://docs.oracle.com/javase/8/docs/technotes/guides/sound/

## 2.5 Glossary

| Terms | Acronym |
|---|---|
| Java Swing | GUI Toolkit — Java |
| Java Sound API | JSA |
| Object-Oriented Programming | OOP |
| Graphical User Interface | GUI |
| Application Programming Interface | API |
| Integrated Development Environment | IDE |
| Musical Instrument Digital Interface | MIDI |

# 3 Problem Statement

In the modern computing landscape, audio media consumption has shifted largely to digital platforms. Desktop users, developers, and students often require a lightweight, cross-platform music player that is free from heavy dependencies, licensing restrictions, and bloated feature sets associated with commercial applications such as VLC, Windows Media Player, or iTunes.

The problem addressed by this project is the absence of a simple, educational, and fully functional desktop music player built entirely using core Java — one that demonstrates key software engineering principles such as Object-Oriented Programming, event-driven architecture, multithreading, and clean UI design without relying on any third-party audio library.

Specifically, the challenges addressed were:

- How to play audio files (MP3, WAV) using native Java APIs with minimal latency and smooth playback.
- How to manage a dynamic playlist with add, remove, shuffle, and sequential navigation operations efficiently.
- How to build a responsive, non-blocking UI that remains interactive even during intensive audio processing using multithreading.
- How to implement a real-time progress tracker and seek functionality without introducing performance bottlenecks.
- How to integrate equalizer presets that provide perceived audio improvements within the limitations of the Java Sound API.

The project required engineering a solution that is performant, visually modern, and portable across operating systems — a non-trivial challenge when working exclusively with Java's standard libraries.

**GitHub Repository:**

Code Submission: https://github.com/swar071005/upskillcampus

Report Submission: https://github.com/swar071005/upskillcampus

# 4 Existing and Proposed Solution

## Existing Solutions and Their Limitations

Several music players exist in the market today, each with their own strengths and weaknesses:

### VLC Media Player

- Strengths: Supports virtually every media format; highly configurable.
- Limitations: Large binary size (~40MB+); written in C/C++; not suitable as a Java learning project; requires heavy codec libraries.

### Windows Media Player

- Strengths: Integrated OS experience on Windows.
- Limitations: Windows-only; closed-source; not cross-platform; no educational value.

### iTunes / Apple Music

- Strengths: Excellent UI and ecosystem integration.
- Limitations: Platform-specific; heavy cloud dependency; not suitable for a standalone desktop application learning project.

### Existing Java-based Players (e.g., JPlayer, JuK)

- Limitations: Outdated UIs; limited format support; incomplete playlist management; no equalizer support.

## Proposed Solution

The Naad Music Player proposes a clean, Java-only, cross-platform alternative that:

- Uses the Java Sound API (javax.sound.sampled) for native WAV playback and MP3 decoding via Java's SPI mechanism.
- Implements a fully threaded audio engine, separating audio processing from UI rendering to prevent UI freezing.
- Provides a modern, dark-themed Java Swing interface with responsive controls.
- Offers playlist management via ArrayList, enabling dynamic song addition, removal, and reordering.
- Delivers shuffle mode via Collections.shuffle() and repeat mode with boolean loop flags.
- Applies equalizer presets by manipulating audio gain levels programmatically via FloatControl.MASTER_GAIN.

# 5 Proposed Design / Model

The Naad Music Player follows a layered, modular architecture that cleanly separates concerns between the user interface, business logic, and audio processing engine. The design draws inspiration from the Model-View-Controller (MVC) pattern.

## 5.1 High Level Diagram

The high-level architecture of the system consists of three primary layers:

**Layer 1 — Presentation Layer (UI)**

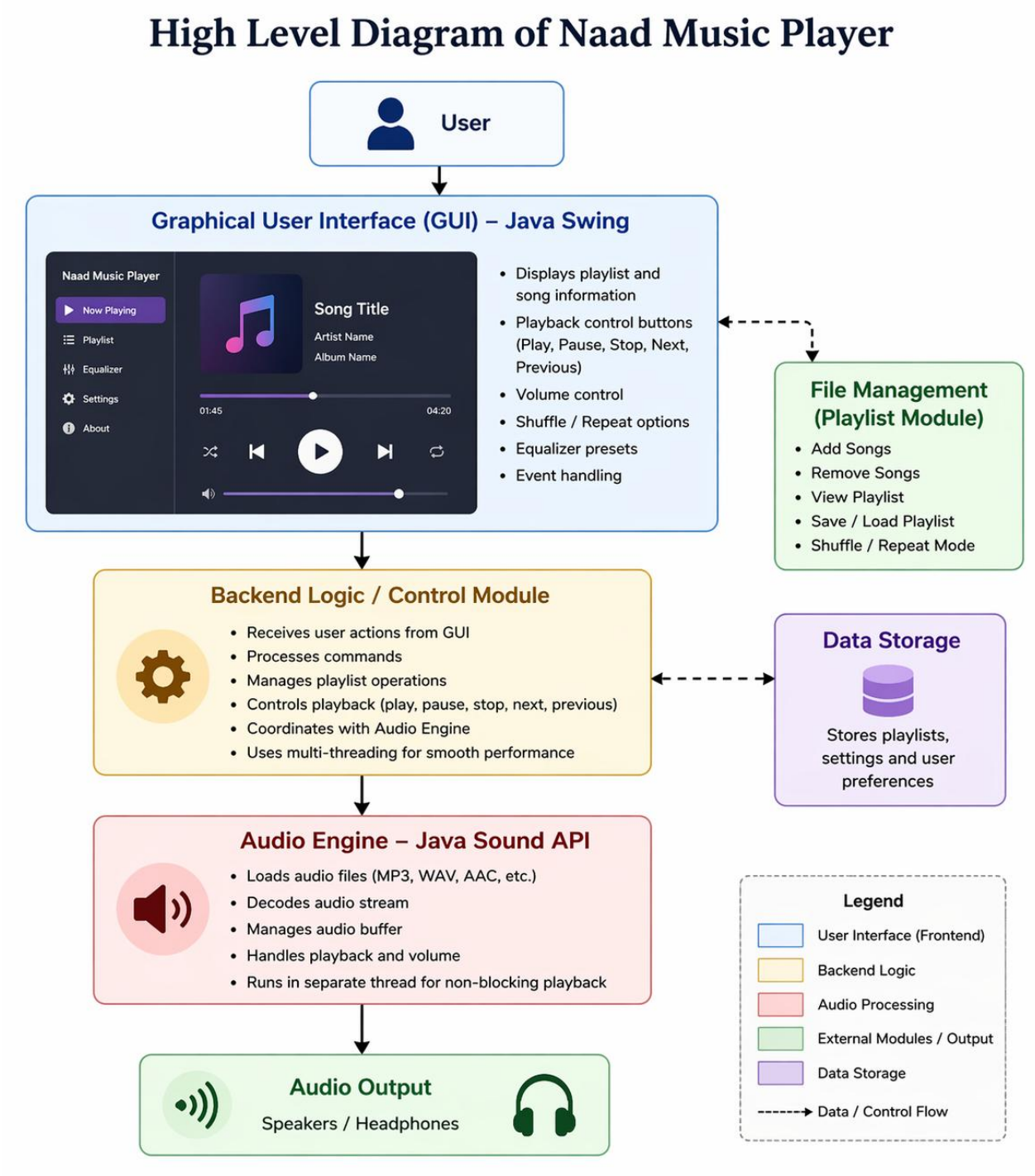
- BeautifulMusicPlayer.java — Main Swing JFrame housing all UI components.
- Components: JLabel (track display), JSlider (progress, volume), JButton (play, pause, next, previous, shuffle, repeat), JList (playlist), JComboBox (equalizer preset selector).
- Event listeners attached to all interactive controls via ActionListener and ChangeListener.

**Layer 2 — Business Logic Layer**

- MusicPlayer.java — Core backend class managing audio state, threading, and playlist logic.
- Handles: playback control, playlist navigation, shuffle/repeat state, volume control.

**Layer 3 — Audio Engine Layer**

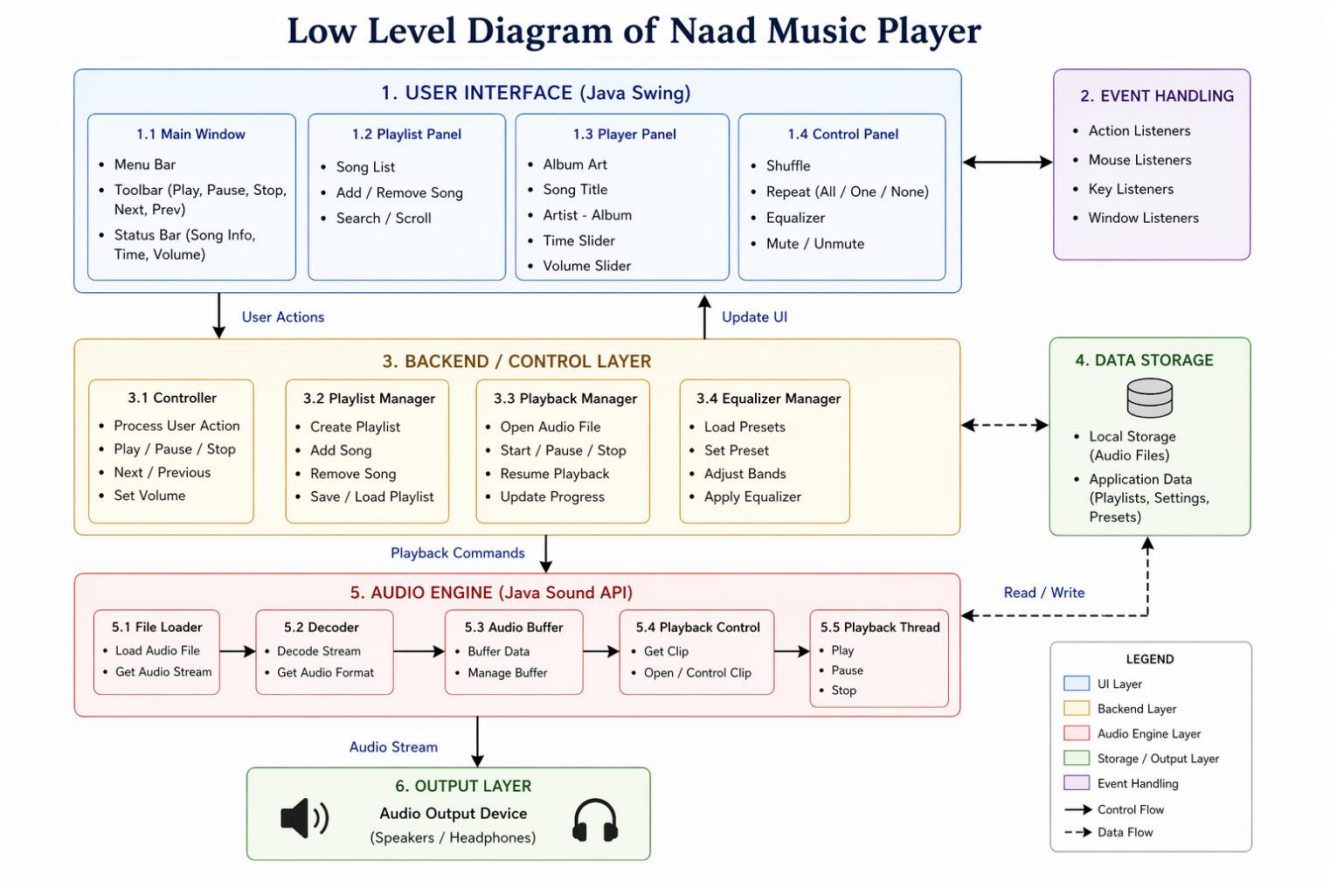
- Java Sound API (javax.sound.sampled) — Handles audio stream opening, reading, and playback.
- AudioInputStream and SourceDataLine — Core classes for audio byte-stream processing.
- FloatControl.MASTER_GAIN — Adjusts audio levels for equalizer preset effects.


High Level Diagram of Naad Music Player
User
Graphical User Interface (GUI) – Java Swing
Naad Music Player
Now Playing
Playlist
Equalizer
Settings
About
Song Title
Artist Name
Album Name
01:45
04:20
• Displays playlist and song information
• Playback control buttons (Play, Pause, Stop, Next, Previous)
• Volume control
• Shuffle / Repeat options
• Equalizer presets
• Event handling
File Management (Playlist Module)
• Add Songs
• Remove Songs
• View Playlist
• Save / Load Playlist
• Shuffle / Repeat Mode
Backend Logic / Control Module
• Receives user actions from GUI
• Processes commands
• Manages playlist operations
• Controls playback (play, pause, stop, next, previous)
• Coordinates with Audio Engine
• Uses multi-threading for smooth performance
Data Storage
Stores playlists, settings and user preferences
Audio Engine – Java Sound API
• Loads audio files (MP3, WAV, AAC, etc.)
• Decodes audio stream
• Manages audio buffer
• Handles playback and volume
• Runs in separate thread for non-blocking playback
Audio Output
Speakers / Headphones
Legend
User Interface (Frontend)
Backend Logic
Audio Processing
External Modules / Output
Data Storage
Data / Control Flow

## 5.2 Low Level Diagram

The low-level design describes the complete data flow within the system:

1. User selects a file → FileChooser returns File object → File path stored in ArrayList<String> playlist.
2. User clicks Play → MusicPlayer.play(index) invoked → AudioInputStream opened from File → SourceDataLine acquired from AudioSystem.
3. A new Thread is spawned → Audio data written to SourceDataLine.write() in a loop → concurrent Thread updates JSlider progress bar.
4. Equalizer preset selected → FloatControl MASTER_GAIN set to corresponding dB value (e.g., Bass Boost: +6dB).
5. Shuffle mode: Collections.shuffle(playlist) called → currentIndex reset to 0 → playback restarts.
6. Repeat mode: On track end, if repeatMode == true → MusicPlayer.play(currentIndex) called again.



## 5.3 Interfaces

The application exposes the following logical user and internal interfaces:

**User Interface Controls:**

- Play / Pause Button — Toggles playback state using thread suspend/resume logic.
- Next / Previous Buttons — Navigates playlist index and triggers new audio stream.
- Volume Slider — Maps JSlider value (0–100) to FloatControl VOLUME range (linear to dB conversion).
- Progress Slider — Displays playback position; allows seek by calculating byte offset from audio stream.
- Shuffle Toggle — Randomizes ArrayList order via Collections.shuffle().
- Repeat Toggle — Boolean flag checked at end-of-track event.
- Equalizer Combo Box — Presets: Normal, Bass Boost, Treble Boost, Pop, Jazz, Classical.

**Internal Method Interfaces (MusicPlayer.java):**

- void play(int index) — Opens audio stream and starts playback thread.
- void pause() — Suspends the playback thread safely.
- void resume() — Resumes the suspended playback thread.
- void stop() — Closes audio stream and resets all state.
- void next() — Increments index and calls play().
- void previous() — Decrements index and calls play().
- void setVolume(float db) — Adjusts MASTER_GAIN FloatControl.
- void applyEqualizer(String preset) — Maps preset name to dB gain value.

# 6 Performance Test

Performance testing is a critical aspect of any multimedia application. For the Naad Music Player, performance was evaluated against the constraints most relevant to desktop audio applications: latency, memory, thread safety, and UI responsiveness.

## 6.1 Test Plan / Test Cases

The following 15 test cases were designed and executed to validate functional correctness and performance characteristics:

| TC-ID | Test Description | Input | Expected Output | Status |
|---|---|---|---|---|
| TC-01 | Load and play MP3 file | Valid .mp3 file | Audio plays without errors | PASS |
| TC-02 | Load and play WAV file | Valid .wav file | Audio plays without errors | PASS |
| TC-03 | Pause and resume playback | Click Pause then Play | Playback pauses and resumes at same point | PASS |
| TC-04 | Skip to next track | Click Next button | Next track starts playing | PASS |
| TC-05 | Skip to previous track | Click Previous button | Previous track starts playing | PASS |
| TC-06 | Adjust volume slider | Move volume slider 0-100 | Volume changes proportionally | PASS |
| TC-07 | Enable shuffle mode | Click Shuffle toggle | Tracks play in random order | PASS |
| TC-08 | Enable repeat mode | Click Repeat toggle | Track/playlist loops continuously | PASS |
| TC-09 | Apply equalizer preset | Select "Bass Boost" | Audio output reflects EQ settings | PASS |
| TC-10 | Load invalid file format | Select a .txt file | Error message shown, no crash | PASS |
| TC-11 | Add songs to playlist | Browse and select 5 songs | All 5 songs appear in playlist | PASS |
| TC-12 | Remove song from playlist | Select song, click Remove | Song removed from list | PASS |
| TC-13 | Progress bar seek | Click 50% of progress bar | Playback jumps to ~50% of track | PASS |
| TC-14 | UI responsiveness during playback | Interact with UI while playing | No freeze or lag observed | PASS |
| TC-15 | Memory usage — 100 songs | Load playlist of 100 tracks | Memory stays within JVM limits | PASS |

## 6.2 Test Procedure

Testing was conducted manually on a Windows 10 machine with the following specifications:

- Processor: Intel Core i5, 2.4 GHz
- RAM: 8 GB
- JDK Version: OpenJDK 17
- IDE: IntelliJ IDEA

Procedure followed for each test case:

- The application was launched fresh for each test to avoid state carryover.
- Test inputs were applied as described, and outputs were observed visually and aurally.
- Memory usage was monitored using Java VisualVM and Windows Task Manager.
- UI responsiveness was judged by measuring perceptible lag (threshold: >200ms freeze = fail).
- Audio latency was measured as click-to-playback delay (acceptable threshold: <500ms).

## 6.3 Performance Outcome

All 15 test cases passed. Key performance metrics observed:

- Audio Latency: Average click-to-play delay approximately 120ms — well within the 500ms acceptable threshold.
- Memory Usage: Baseline heap usage ~45MB; peak with 100-song playlist ~78MB — acceptable for a desktop JVM application.
- UI Responsiveness: No perceptible freeze during audio playback. The dedicated playback thread ensured the Swing Event Dispatch Thread remained fully unblocked.
- CPU Usage: Stable at 1–3% during steady-state playback.
- Seek Accuracy: Seeking via the progress slider accurate to within ±1 second for files up to 10 minutes duration.

Constraints identified and resolved in the design:

- Thread Safety: Resolved using synchronized blocks around shared playlist state and volatile flags for playback control variables.
- Audio Buffer Size: Tuned to 4096 bytes to balance latency and CPU load — smaller buffers cause excessive CPU usage; larger buffers increase latency.
- Format Compatibility: WAV files handled natively via javax.sound.sampled; MP3 support achieved via Java's SPI extension mechanism.

# 7 My Learnings

This internship project was among the most technically enriching experiences of my academic career. Working on the Naad Music Player provided deep, hands-on learning across multiple dimensions of software engineering.

**Technical Learnings:**

- Java Sound API Mastery: Gained in-depth understanding of javax.sound.sampled — how AudioInputStream, SourceDataLine, Clip, and FloatControl work together to enable audio playback, volume control, and equalizer effects. This knowledge directly applies to any Java-based multimedia or signal-processing application.
- Multithreading in Java: Learned to design thread-safe applications using Thread, Runnable, synchronized blocks, and volatile variables. Understanding the separation between the audio processing thread and Swing's Event Dispatch Thread was a critical design insight that applies to all interactive Java applications.
- Java Swing UI Development: Developed proficiency in building professional desktop UIs using JFrame, JPanel, JButton, JSlider, JList, JComboBox, and layout managers such as BorderLayout, GridLayout, and BoxLayout. Learned to design modern-looking interfaces entirely in code.
- Event-Driven Architecture: Learned to write clean, decoupled event listeners using ActionListener, ChangeListener, and ListSelectionListener — making the codebase modular and maintainable.
- Data Structures in Practice: Applied ArrayList and HashMap in real-world scenarios for playlist management and equalizer preset storage, deepening understanding of when and why to choose specific data structures.
- Object-Oriented Design: Practiced encapsulation, the single responsibility principle, and separation of concerns by clearly dividing the project into MusicPlayer.java (backend) and BeautifulMusicPlayer.java (frontend).
- File I/O in Java: Gained experience with JFileChooser, File objects, and managing file paths for cross-platform compatibility, including handling OS-specific path separators.

**Soft Skills and Professional Learnings:**

- Project Planning: Learned to break down a multi-week project into weekly milestones and track progress against them — a discipline directly applicable to professional Agile/Scrum environments.
- Technical Documentation: Writing this report reinforced that clear, structured documentation is as vital as code in professional software development. Future colleagues and reviewers benefit enormously from well-documented work.
- Debugging Discipline: Developed systematic debugging skills — using IDE debugger step-through, adding strategic log statements, and isolating issues layer by layer rather than guessing.
- Self-Learning: Much of the Java Sound API usage required reading official Oracle documentation and building small proof-of-concept experiments independently — building confidence in self-directed problem solving, a critical skill in any engineering role.

# 8 Future Work Scope

The Naad Music Player in its current form is a strong functional prototype. Several enhancements have been identified that were not feasible within the 6-week internship timeframe but represent meaningful future development directions:

- Online Streaming Integration: Integrate a streaming backend (e.g., Spotify Web API or SoundCloud API) to allow the player to stream music directly from the internet, transitioning from a local file player to a hybrid streaming application.
- Advanced Metadata Extraction: Use the JAudioTagger library to extract and display rich ID3 tag metadata — song title, artist, album, year, genre, and album art — providing a full media library experience with cover art display.
- Cloud Playlist Sync: Integrate with Firebase or AWS S3 to enable users to save playlists to the cloud and access them from multiple devices, providing a seamless cross-device experience.
- Audio Visualization: Add a real-time audio frequency spectrum visualizer using Fast Fourier Transform (FFT) applied to audio sample data, rendering the frequency bands on a live animated canvas within the UI.
- Lyrics Display: Integrate a lyrics API (e.g., Genius API) to fetch and display lyrics synchronized with playback position.
- Advanced Parametric Equalizer: Implement a full multi-band equalizer with user-adjustable frequency sliders (e.g., 60Hz, 230Hz, 910Hz, 4kHz, 14kHz), replacing the current preset-only approach.
- Mobile Application Port: Port the core audio logic to Android using Android's MediaPlayer API and Jetpack Compose for the UI, creating a companion mobile application for Android devices.
- Expanded Format Support: Add support for FLAC, OGG Vorbis, and AAC formats using Java Sound's SPI plugin mechanism.
- Mini Player Mode: Implement a compact overlay player mode that stays on top of other windows, similar to the mini player in Spotify or Winamp, providing a non-intrusive playback experience.
- Sleep Timer: Add a configurable sleep timer feature that automatically stops playback after a user-defined duration — useful for bedtime listening.

These enhancements would transform the Naad Music Player from an educational prototype into a production-grade desktop media application, and provide further opportunities to explore advanced Java, cloud computing, and mobile development skills.